Network Anomaly Detection Using Quantum Neural Networks

Chiara Peppicelli
chiara.peppicelli@edu.unifi.it
Mat: 7146573

Abstract

Network anomaly detection is a critical component of modern cybersecurity systems. Recent developments in quantum machine learning suggest that Quantum Neural Networks (QNNs) may offer advantages in processing structured and high-dimensional data, even when executed on Noisy Intermediate-Scale Quantum (NISQ) devices.

This report presents an implementation and analysis of the methodology proposed in "Network Anomaly Detection Using Quantum Neural Networks on Noisy Quantum Computers" [1]. I reproduced the core components of the original study, including the model architectures, classical-to-quantum encoding strategy, and the certainty factor used to assess prediction confidence. The experimental evaluation is conducted on a network intrusion dataset, examining both classification performance and robustness under noise conditions. The experiments show that QNN-based models can achieve competitive anomaly detection performance in noiseless simulations, while their effectiveness degrades under increasing noise, highlighting current limitations of NISQ hardware. The analysis of the certainty factor further provides insight into prediction reliability and noise sensitivity. The implementation and experimental results are publicly available for reproducibility and further investigation (`https://github.com/chiarapepp/Quantum-Machine-Learning-Project`).

Future Distribution Permission

The author of this report give permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

The increasing number of network based attacks has made intrusion detection a critical component of modern cybersecurity systems. Traditional intrusion detection systems (IDS) often struggle to cope with the growing complexity and scale of network traffic, motivating the adoption of machine learning (ML) techniques in network intrusion detection systems (NIDS). By learning patterns directly from network data, ML based approaches can improve the detection of anomalous or malicious network activities.

Recent advances in quantum computing have introduced new possibilities in the field of quantum machine learning (QML). In particular, parameterized quantum circuits and variational algorithms have enabled the development of quantum classifiers such as Quantum Neural Networks (QNNs), which are increasingly explored for classification tasks.

However, current quantum hardware operates in the noisy intermediate-scale quantum (**NISQ**) era, where noise and hardware limitations significantly affect algorithmic performance. While several studies have reported promising results for quantum classifiers in noiseless simulations, achieving reliable performance on real quantum hardware remains a challenge.

Kukliansky et al. [1] investigated the use of QNNs for **network anomaly detection** and evaluated several QNN architectures for this task. Their study compares different quantum circuit models and shows that a simplified ultralean architecture can achieve competitive performance; in addition, their work introduces an efficient encoding scheme for representing NetFlow data, as well as the introduction of a certainty factor to analyze the confidence of classification predictions.

This project focuses on the reimplementation and experimental analysis of the proposed approach using the **PennyLane quantum machine**

learning framework. The implementation includes the classical preprocessing pipeline, the feature encoding strategy, and the construction and training of the parameterized quantum circuits used for anomaly detection.

1.1. Dataset

The experiments in this project are based on the **NF-UNSW-NB15** dataset, a NetFlow-based version of the widely used UNSW-NB15 intrusion detection benchmark. The dataset provides a large and diverse collection of labeled network flows representing both benign activity and multiple types of malicious behavior, making it suitable for evaluating anomaly detection models under realistic conditions.

The dataset contains 2,390,275 NetFlow entries and 12 features. Each sample is labeled either 0 (benign) and 1 (malicious). Following the methodology described in the referenced study, features related to source and destination IP addresses and ports were removed in order to reduce the dimensionality of the data and avoid potential overfitting. This preprocessing step resulted in a reduced set of **eight numerical features** describing statistical properties of the network flows (see Figure 1). These features are used as input to the quantum machine learning pipeline implemented in this work.

Feature Name	Type	Range	Unique Values
IP Protocol	int	0 to 255	255
TCP Flags	int	0 to 31	15
Layer 7 Protocol	float	0 to 249	265
In Byte Count	int	52 to $2.68 \cdot 10^7$	12256
Out Byte Count	int	0 to $1.46 \cdot 10^7$	16047
In Packet Count	int	1 to 20038	892
Out Packet Count	int	0 to 11024	1207
Flow Duration [ms]	int	0 to $4.3 \cdot 10^6$	17377

Figure 1: Selected features from the NF-UNSW-NB15 dataset, including the range of each feature and the number of unique values.

As is typical for network traffic datasets, the class distribution is highly imbalanced, with 2,295,222 benign samples and 95,053 malicious samples. To address this imbalance, benign samples were randomly **downsampled** to match the number of malicious samples using scikit-learn's `resample` function (`random_state=123`). This resulted in a balanced dataset of 190,106 NetFlow records (95,053 benign and 95,053 malicious samples).

The numerical difference with respect to the balanced size reported in the reference paper arises from the use of a different release of the NF-UNSW-NB15 dataset, while the balancing methodology remains identical.

Following preprocessing and class balancing, the dataset was split into training and test sets using an 85%–15% ratio with stratified sampling. The resulting partitions consist of:

- Training set: 161,590 samples;
- Test set: 28,516 samples.

In addition 15% of the training portion is used as a validation set.

2. Methodology

The methodology followed in this project replicates the experimental pipeline proposed in the reference study. Classical features are first prepared and encoded into quantum states, after which several quantum neural network (QNN) architectures are trained and evaluated for the task of network anomaly detection.

2.1. Data Encoding

In order to process classical network-flow features within a Quantum Neural Network (QNN), each attribute is mapped into a quantum state through a **one-qubit-per-feature** encoding scheme. Each input feature is associated with a dedicated qubit initialized in the $|0\rangle$ state.

The feature value is embedded into the qubit via a rotation around the x-axis:

$$|\psi_i^{[j]}\rangle = R_x(\theta_i^{[j]})|0\rangle = \cos\left(\frac{\theta_i^{[j]}}{2}\right)|0\rangle + \sin\left(\frac{\theta_i^{[j]}}{2}\right)|1\rangle$$

where $\theta_i^{[j]} \in [0, \pi]$ represents the rotation angle corresponding to feature j of sample i.

This encoding establishes a direct correspondence between the eight selected features of the NF-UNSW-NB15 dataset and the eight qubits used in the quantum circuit. The one-qubit-per-feature approach avoids complex state preparation procedures, thereby reducing circuit depth and limiting the impact of noise, which is crucial for NISQ devices.

The mapping from feature values to rotation angles depends on the statistical properties of each feature. For features with a limited number of distinct values, angles are assigned categorically within the available range. For features with a large number of unique values, the mapping is performed using percentile-based binning, ensuring a balanced distribution of angles across $[0, \pi]$.

1. **Features with a small number of unique values**

   Attributes such as the IP protocol field and TCP flags contain a limited set of distinct values. For these variables, a categorical mapping is used in which each unique value is assigned an evenly spaced angle within the interval $[0, \pi]$.

   If a feature has N unique values, the corresponding rotation angles are defined as

   $$\theta_k = \frac{k}{N-1}\,\pi, \qquad k = 0, \ldots, N-1.$$

   This mapping preserves the discrete structure of the feature while producing deterministic and noise-resilient rotations.

2. **Continuous numerical features** Flow-level metrics such as byte counts, packet counts, and flow duration exhibit wide numeric ranges and often skewed distributions. To encode these variables, a percentile-based binning strategy is adopted to ensure a balanced use of the angular range.

   Specifically, the interval $[0, \pi]$ is divided into B bins, corresponding to equally spaced percentiles of the empirical feature distribution. Each sample is assigned to a bin according to its percentile rank. For example, when $B = 100$, a feature value located at the x-th percentile is mapped to the angle

   $$\theta = \frac{x}{100}\,\pi.$$

   This approach preserves the relative ordering of feature values while avoiding excessive concentration of angles in densely populated regions, leading to a more uniform and noise-robust encoding.

After computing the raw angles using either the categorical or percentile based mapping, all values are **quantized** to the nearest multiple of the **0.25**° rotation step. This ensures robustness against hardware imperfections since this discretization limits the precision required for quantum gate rotations and improves stability under realistic noise conditions.

For each dataset sample, the final output of the encoding stage is an eight dimensional vector of rotation angles one for each feature. These angles are then used to apply the sequence of encoding gates:

$$R_x(\theta_1),\ R_x(\theta_2),\ \ldots,\ R_x(\theta_8)$$

on the corresponding eight qubits at the input layer of the QNN.

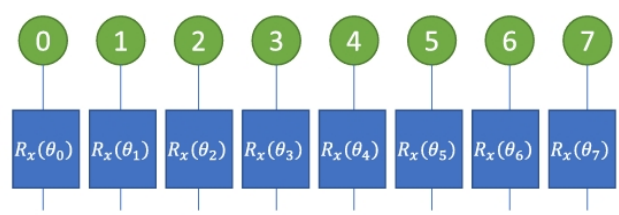

Figure 2: State preparation circuit used for the qubit encoding of the classical features.

2.2. QNN Architecture

To investigate the suitability of quantum machine learning models for network anomaly detection, four different Quantum Neural Network (QNN) architectures are evaluated: a Simple variational circuit, a Tree Tensor Network

(TTN), a Multi-scale Entanglement Renormalization Ansatz (MERA), and a Quantum Convolutional Neural Network (QCNN). These architectures differ in their connectivity patterns, parameter-sharing mechanisms, and entanglement structures, enabling a **comparative analysis of expressiveness and circuit complexity**.

All architectures operate on the same encoded input features described in the previous section and produc e a **single-qubit output**, which is measured to obtain the final binary classification result.

Generic Two-Qubit Building Block

The TTN, MERA, and QCNN architectures are constructed from a common primitive: a generic two qubit gate composed of two parameterized single qubit rotations followed by a Controlled NOT (CNOT) gate.

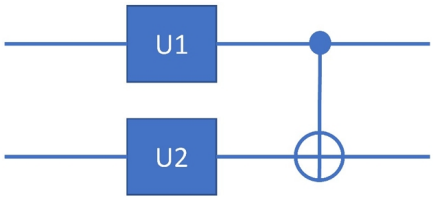

Figure 3: Two-qubit gate modeled by two generic one-qubit rotations followed by a CNOT gate.

Each single qubit operation is implemented as a general rotation gate with three trainable parameters, allowing arbitrary rotations on the Bloch sphere. Consequently, each two qubit block contains **six trainable parameters** in total. The entangling operation is then introduced through a single **CNOT gate**.

Although an exact decomposition of a general two qubit unitary requires three CNOT gates, the reference study intentionally restricts the circuit to a single CNOT in order to reduce circuit depth and improve robustness on noisy intermediate scale quantum devices. This design represents a compromise between expressibility and hardware feasibility.

TTN Architecture

The TTN architecture implements a hierarchical tree structure that progressively reduces the number of active qubits.

At each layer, adjacent qubit pairs are processed using the generic two-qubit block described earlier. After the interaction, only one qubit from each pair is retained for the next level of the tree, effectively reducing the number of active qubits by approximately half at each step.

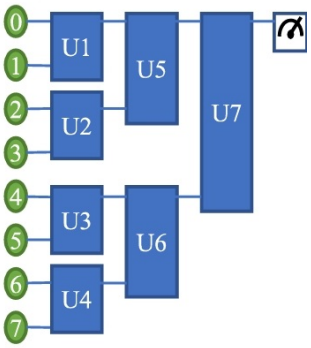

Figure 4: TTN Architecture.

This hierarchical contraction results in:

- **Logarithmic circuit depth**, improving scalability for larger input sizes. Although in this implementation the number of input features is fixed, this property remains relevant for potential extensions to higher-dimensional data.
- **Localized entanglement**, capturing correlations between neighboring features.
- **Reduced active qubit count** in deeper layers, which may improve robustness to noise.

The process continues until a single qubit remains, which serves as the output of the network. This qubit is measured in the **Z-basis**.

MERA Architecture

The MERA architecture extends the TTN structure by introducing an additional disentangling stage before each coarse graining step. At a given scale, the architecture performs the following sequence of operations:

- **Disentanglers**: the parametrized two-qubit block is applied to **shifted neighboring pairs**

$$(q_1, q_2),\ (q_3, q_4),\ \ldots$$

  This step redistributes correlations across adjacent qubits before the reduction stage.

- **Isometries**: the two-qubit block is then applied to the **even pairs**

$$(q_0, q_1),\ (q_2, q_3),\ \ldots$$

  These operations combine the information stored in neighboring qubits and prepare the system for hierarchical reduction.

This design makes the MERA architecture more expressive than TTN: while the isometries perform the same hierarchical reduction as in a tree structure, the preliminary disentangling stage allows correlations to be redistributed before qubit reduction, improving the model's ability to capture dependencies across multiple scales.

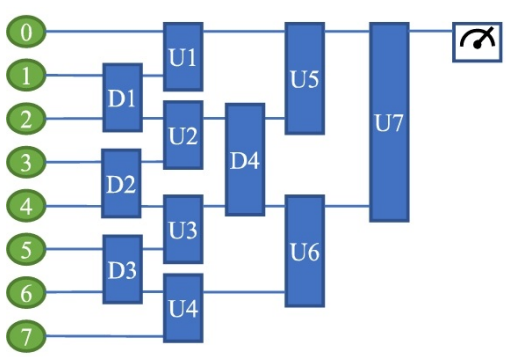

Figure 5: MERA Architecture.

The final qubit is measured in the **Z-basis** to obtain the classification result.

Quantum Convolutional Neural Network (QCNN)

The QCNN architecture adapts principles from classical convolutional neural networks to the quantum domain. It consists of alternating layers of:

- **Quantum convolution layers**, where the generic two-qubit block is applied to neighboring qubit pairs.
- **Quantum pooling layers**, which reduce the number of active qubits by combining information from pairs of qubits.

A key characteristic of QCNN is **parameter sharing**: all gates occupying the same relative position within a convolution layer share identical parameters. This mechanism reduces the number of trainable parameters and introduces translation-invariant processing across the qubit register.

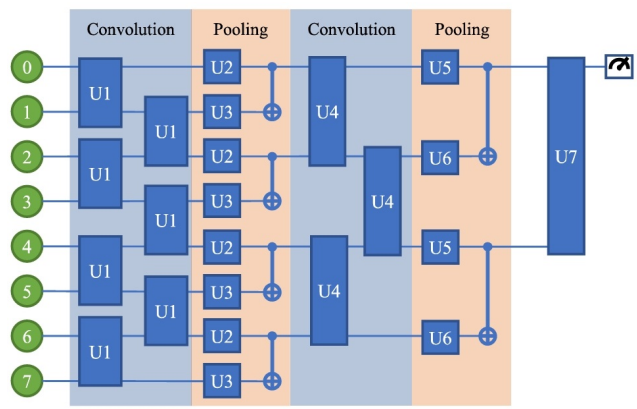

Figure 6: QCNN Architecture.

After each convolution–pooling stage, the number of active qubits decreases until only two remain. A final classification block is then applied, and the resulting output qubit is measured in the **Z-basis**.

Simple Architecture

The Simple architecture follows a different design principle compared to the other models. Instead of using a generic two-qubit block, it is constructed using parametrized **Pauli interaction rotations** between a dedicated output qubit and the feature qubits.

In addition to the feature qubits, the circuit introduces a **result qubit**, which serves as the output of the model. This qubit is initialized in the $|-\rangle$ state by applying a Pauli-X gate followed by a Hadamard gate:

$$|-\rangle = HX|0\rangle.$$

After encoding the classical features into the feature qubits, the circuit applies a sequence of interaction layers. Each layer is implemented through a nested loop structure in which:

- a set of Pauli interaction types is selected (e.g., XX, YY, ZZ),
- for each interaction type, the corresponding two-qubit gate is applied between the result qubit and **every feature qubit**.

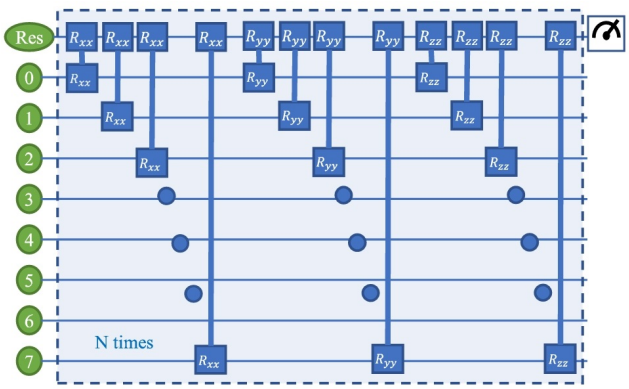

Figure 7: Simple Architecture.

Concretely, for each layer and each interaction type, the following gates are applied: $R_{XX}(\theta)$, $R_{YY}(\theta)$, $R_{ZZ}(\theta)$ where each interaction is applied between the result qubit and a single feature qubit, forming a **star-shaped connectivity** centered on the result qubit. Importantly, **each gate is associated with an independent trainable parameter**, meaning that parameters are not shared across different qubits or interaction types.

Different layer configurations are obtained by selecting specific sequences of interaction types. For example, a configuration such as `XXYY` applies a full sweep of R_{XX} interactions across all feature qubits, followed by a sweep of R_{YY} interactions. More complex configurations such as `ZZXXYY` include all three interaction types within each layer.

At the end of the circuit, the result qubit is measured in the **X-basis**.

Readout and Decision Rule

All considered QNN architectures produce a single scalar output obtained from the expectation value of a Pauli observable measured on the final readout qubit.

- In the **Simple** architecture, the dedicated result qubit is initialized in the $|-\rangle$ state and the model output is the expectation value $\langle X \rangle$ measured on that qubit.
- In the **TTN**, **MERA**, and **QCNN** architectures, the output is the expectation value $\langle Z \rangle$ measured on the final surviving qubit.

Therefore, independently of the architecture, the classifier returns a real-valued score

$$s(x) \in [-1, 1],$$

where $s(x)$ denotes either $\langle X \rangle$ or $\langle Z \rangle$ depending on the chosen circuit.

The sign of the score is used for binary classification. Since training is performed with hinge loss on labels encoded as $\{-1, +1\}$, the decision rule is

$$\hat{y} = \begin{cases} +1\,\texttt{(malicious)} & \text{if } s(x) \geq 0, \\ -1\,\texttt{(benign)} & \text{if } s(x) < 0. \end{cases}$$

This readout strategy is fully consistent with the training objective: positive outputs are encouraged for samples belonging to the positive class, while negative outputs are encouraged for samples belonging to the negative class.

2.3. Certainty Factor in the Implemented Model

Beyond its role in binary classification, the model output is also interpreted as a continuous indicator of decision strength. The reference paper introduces the **certainty factor** as a signed quantity in the interval $[-1, 1]$, intended to capture how strongly the predicted quantum state supports one class over the other.

In the implementation, the certainty factor is identified directly with the model score:

$$C(x) = s(x).$$

This choice is natural because the expectation value already provides a signed continuous output in the same range:

- $C(x) > 0$ indicates support for the positive class,
- $C(x) < 0$ indicates support for the negative class,
- $|C(x)| \approx 1$ corresponds to a sharp and confident decision,
- $|C(x)| \approx 0$ indicates an uncertain prediction close to the decision boundary.

Accordingly, the absolute value

$$|C(x)|$$

can be interpreted as a confidence score, while the sign preserves the predicted class direction. This interpretation is particularly useful when analyzing borderline samples and when later studying the degradation of predictions under noisy evaluation.

3. Experiments and Results

This section presents the experimental evaluation of the implemented quantum models for network anomaly detection. The experiments were designed to reproduce the main analyses of the reference paper, with particular attention to the comparison of different QNN architectures in the noiseless setting and to the study of robustness under noisy simulation.

All experiments were carried out using the PennyLane framework. The TTN, MERA, QCNN, and Simple architectures described in the previous section were trained on the encoded NF-UNSW-NB15 dataset using gradient-based optimization. Multiple runs were performed for each model by varying the main hyperparameters, including optimizer, learning rate, batch size, and, when applicable, circuit depth and layer configuration.

Model selection was based on validation performance, while final results were reported on the test set. The main evaluation metrics considered in this work are accuracy, precision, recall, F1-score, and ROC-AUC. In addition, the raw circuit outputs were retained in order to analyze the certainty factor and its behavior under noise.

The experimental analysis is organized as follows. First, the architectures are compared in the noiseless setting. Then, a more detailed analysis of the best-performing Simple architecture is presented. Finally, the effect of noise on classification performance and prediction certainty is discussed.

3.1. Noiseless Architecture Comparison

The first group of experiments was conducted in the noiseless setting using ideal quantum simulation. In this setting, the output expectation values are computed analytically from the simulated quantum state, without hardware noise and without finite-shot measurement fluctuations. This makes it possible to evaluate the intrinsic classification capability of the considered QNN architectures under ideal conditions.

The comparison includes the four architectures introduced in the previous section. For each model, multiple training runs were performed by *varying the main hyperparameters*. These include the optimizer, learning rate, and batch size for all architectures and, in the case of the Simple model, also the number of layers and the entangling layer type. Within each run, the best model parameters were selected according to the lowest validation loss, and the corresponding performance on the test set was used for comparison.

Table 1: Best noiseless accuracy and F1-score obtained for each architecture.

Architecture	Test Acc.	F1-score
TTN	0.7430	0.7373
MERA	0.7445	0.7392
QCNN	0.7641	0.7640
Simple	**0.7805**	**0.7972**

Following the evaluation strategy adopted in the reference paper, particular attention is given

to the F1-score, since it provides a more informative assessment than accuracy alone in anomaly-detection tasks. In addition to F1-score, the evaluation pipeline also computes accuracy, precision, recall, and ROC-AUC.

Table 2: Best noiseless precision, recall, and ROC-AUC obtained for each architecture.

Architecture	Precision	Recall	ROC-AUC
TTN	0.7540	0.7214	0.7055
MERA	0.7547	0.7243	0.7114
QCNN	**0.7642**	0.7637	0.7654
Simple	0.7411	**0.8625**	**0.8057**

The results show that the **Simple architecture** achieved the best overall performance in the noiseless setting. In particular, it obtained the highest F1-score 1, equal to 0.7972, together with the highest test accuracy and ROC-AUC among the considered models. This is qualitatively consistent with the trend reported in the reference paper, where the Simple architecture also emerged as the strongest model in noiseless simulation.

Among the remaining architectures, **QCNN** provided the best performance, with a more balanced trade-off between precision and recall. By contrast, **TTN** and **MERA** achieved similar results, with MERA only slightly improving over TTN. In the present implementation, the additional expressive structure of MERA therefore does not lead to a substantial performance gain on this task.

A closer inspection of the best Simple run shows that it was obtained using the Adam optimizer, learning rate 0.05, batch size 32, six layers, and the `XXYY` interaction pattern. This result is strongly aligned with the findings reported in the reference paper.

3.2. Hyperparameter Exploration of the Simple Architecture

Since the Simple architecture achieved the best overall performance in the noiseless comparison, a more detailed analysis was carried out on its **hyperparameter configurations**. This is also consistent with the reference paper, which further investigates the behavior of the Simple model after identifying it as the strongest architecture in the noiseless setting.

The explored runs differ in terms of interaction pattern, circuit depth, optimizer, learning rate, and batch size. In order to better understand the contribution of these factors, the results are reported through a set of complementary tables. The first analysis focuses on the structural properties of the circuit, while the following ones examine the optimization setup in greater detail.

Table 3: Best and mean noiseless performance of the Simple architecture as a function of circuit depth.

Layers	Best Acc.	Best F1	Mean F1
1	0.7642	0.7641	0.7442
2	0.7628	0.7748	0.7445
4	0.7489	0.7691	0.7525
6	**0.7805**	**0.7972**	**0.7596**

Table 3 summarizes the effect of **circuit depth** on the noiseless performance of the Simple architecture. For each depth, the best and mean values are computed over multiple experiments performed with different hyperparameter settings. The results show that the strongest performance is obtained with the deepest tested configuration. In particular, the six-layer Simple circuits provide the highest test accuracy and F1-score. This behavior is consistent with the reference paper, where the best noiseless Simple model is also obtained with a six-layer configuration.

Table 4: Best and mean noiseless performance of the Simple architecture for different interaction patterns.

Layer type	Best Acc.	Best F1	Mean F1
XXYY	**0.7805**	**0.7972**	**0.7606**
ZZXX	0.7642	0.7641	0.7566
ZZYY	0.7596	0.7691	0.7320
ZZXXYY	0.7642	0.7641	0.7343

The effect of the **interaction pattern** is reported in Table 4. Among the tested configurations, `XXYY` clearly emerges as the strongest one, both in terms of best performance and average F1-score across runs. This is in clear agreement with

the reference paper, where the best noiseless performance is also associated with the XXYY pattern.

A more detailed **structural comparison** is reported in Table 5. The strongest overall configuration is the six-layer XXYY model, which substantially improves over the shallower alternatives. This result closely matches the main architectural finding of the paper: the best noiseless Simple classifier is obtained with a relatively deep circuit based on XXYY-type interactions.

Table 5: Best noiseless results for the Simple architecture under different structural configurations.

Layer type	Layers	Test Acc.	F1-score
XXYY	6	**0.7805**	**0.7972**
XXYY	2	0.7628	0.7748
ZZYY	4	0.7489	0.7691
XXYY	1	0.7642	0.7641
ZZXXYY	1	0.7642	0.7641
ZZXX	1	0.7642	0.7641
ZZYY	2	0.7596	0.7630
ZZXX	2	0.7579	0.7609

Table 6: Best noiseless performance for the six-layer XXYY Simple architecture under different optimizers and learning rates.

Optimizer	LR	Best Acc.	Best F1
Adam	0.001	0.7692	0.7783
Adam	0.005	0.7425	0.7613
Adam	0.010	0.7606	0.7723
Adam	0.015	0.7657	0.7741
Adam	0.020	0.7694	0.7785
Adam	0.050	**0.7805**	**0.7972**
Adam	0.100	0.7721	0.7822
SGD	0.001	0.7050	0.7211
SGD	0.005	0.7487	0.7623
SGD	0.010	0.7564	0.7675
SGD	0.015	0.7346	0.7502
SGD	0.020	0.7575	0.7688
SGD	0.050	0.7553	0.7667
SGD	0.100	0.7583	0.7702

After identifying the best structural configuration, the analysis was extended to the **optimization setup**. In particular, the six-layer XXYY model was examined under different **optimizers** and **learning rates**.

Table 6 shows that the best result is achieved with the **Adam** optimizer and learning rate 0.05. More generally, Adam provides stronger performance than SGD across the tested learning rates for this structural configuration. Therefore, while the exact optimization setup differs from the one highlighted in the reference paper, the structural conclusion remains unchanged.

Finally, the effect of **batch size** was analyzed for the same six-layer XXYY configuration.

Table 7: Best noiseless performance for the six-layer XXYY Simple architecture as a function of optimizer and batch size.

Optimizer	Batch size	Best Acc.	Best F1
Adam	16	0.7721	0.7822
Adam	32	0.7805	0.7972
SGD	16	0.7583	0.7702
SGD	32	0.7575	0.7688

As shown in Table 7, batch size 32 yields the strongest result in combination with Adam, while the difference between the two tested batch sizes is smaller for SGD. Overall, the best noiseless configuration found in the present implementation uses the Adam optimizer, learning rate 0.05, batch size 32, six layers, and the XXYY interaction pattern. Overall, these results indicate that the Simple architecture is the strongest model among the tested QNNs in the noiseless setting. In addition, the best performance is obtained with a six-layer XXYY configuration, which is consistent with the main structural trend reported in the reference paper.

3.3. Certainty-Factor Analysis in the Noiseless Setting

To complement the standard classification metrics, the certainty factor was analyzed on the noiseless test outputs of all considered architectures: TTN, MERA, QCNN, and the best Simple configuration. In the **noiseless setting**, the certainty factor coincides with the **signed classifier output** and therefore takes values in the interval

$[-1, 1]$. Values far from zero correspond to more decisive predictions, whereas values close to zero indicate samples located near the decision boundary. The violin plots and histograms provide a direct view of how each model distributes its predictions in terms of certainty and correctness.

A common trend is observed across all four architectures. The certainty distributions are *clearly separated from zero*, in all cases, correctly classified samples are mainly associated with positive certainty values, while incorrect predictions are concentrated on the negative side. This confirms that the certainty factor is consistent with the decision rule of the classifier and provides a meaningful description of prediction sharpness.

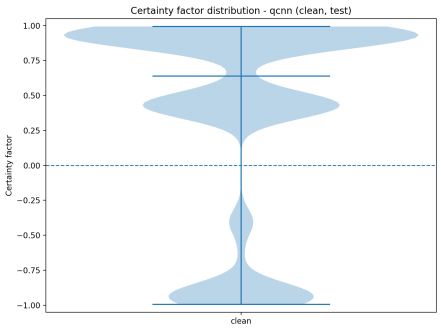

Figure 8: Violin plot of the certainty-factor distribution for the QCNN architecture in the noiseless test setting.

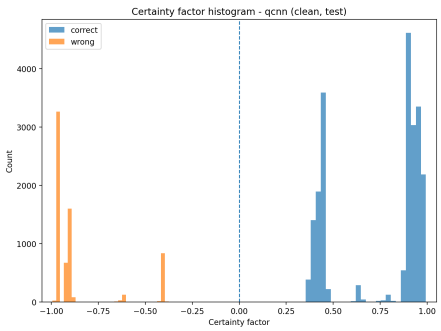

Figure 9: Histogram of the certainty-factor distribution for the QCNN architecture in the noiseless test setting.

QCNN shows the sharpest certainty distribution overall. The fraction of low-certainty samples is almost negligible: no sample satisfies $|C(x)| < 0.1$, only 0.01% satisfies $|C(x)| < 0.2$, and 29.38% satisfies $|C(x)| < 0.5$. This behavior is reflected in the violin plot, where the distribution appears strongly polarized, and in the histogram, where correct predictions are concentrated at high positive certainty and incorrect ones at strongly negative certainty.

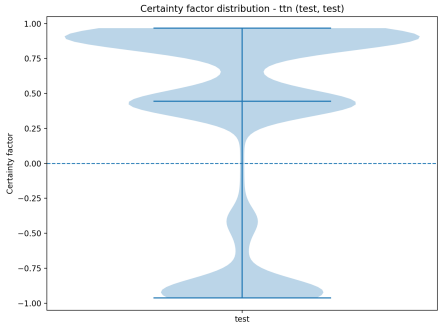

Figure 10: Violin plot of the certainty-factor distribution for the TTN architecture in the noiseless test setting.

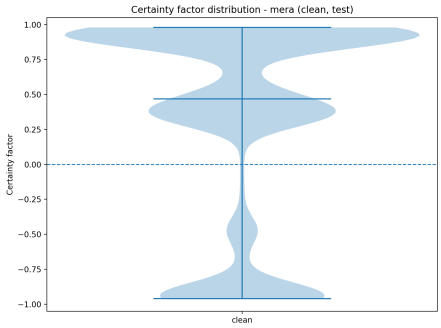

Figure 11: Violin plot of the certainty-factor distribution for the MERA architecture in the noiseless test setting.

TTN and MERA exhibit very similar certainty profiles. MERA reaches a mean absolute certainty of 0.7476, while TTN reaches 0.7420. Their fractions of low-certainty samples are also almost identical. For MERA, 0.43% of the samples satisfy $|C(x)| < 0.1$, 0.83% satisfy $|C(x)| < 0.2$, and 31.54% satisfy $|C(x)| < 0.5$. For TTN, the corresponding values are 0.42%, 0.87%, and 31.58%. The violin plots and histograms confirm this similarity: both architectures show a broad positive region associated with correct predictions and a negative lobe associated with errors. Overall, both models appear comparably sharp and comparably distant from the decision boundary.

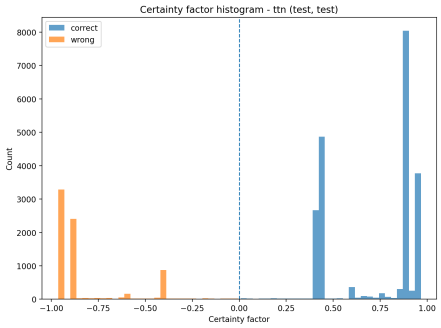

Figure 12: Histogram of the certainty-factor distribution for the TTN architecture in the noiseless test setting.

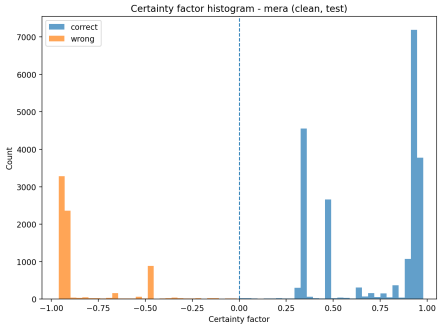

Figure 13: Histogram of the certainty-factor distribution for the MERA architecture in the noiseless test setting.

The best Simple model, namely the six-layer XXYY configuration, shows a less polarized but still well-structured distribution. It achieves a mean absolute certainty of 0.6466. Although its average certainty magnitude is lower than that of QCNN, MERA, and TTN, the distribution remains clearly separated from zero. In particular, 5.68% of the test samples satisfy $|C(x)| < 0.1$, 10.10% satisfy $|C(x)| < 0.2$, and 24.45% satisfy $|C(x)| < 0.5$. Compared with the other models, the Simple architecture distributes more mass over intermediate positive values rather than concentrating it only near the extremes. The histogram is consistent with this pattern: correct predictions remain mostly positive and incorrect ones negative, but both groups are more spread out.

These results highlight that higher certainty does not necessarily imply better classification performance. QCNN has the largest mean absolute certainty, and TTN and MERA also produce sharper outputs on average than the Simple model. Nevertheless, the Simple architecture achieves the best predictive performance, with the highest recall, F1-score, and ROC-AUC. The certainty factor should therefore be interpreted as a measure of decisiveness rather than as a direct proxy for classification quality.

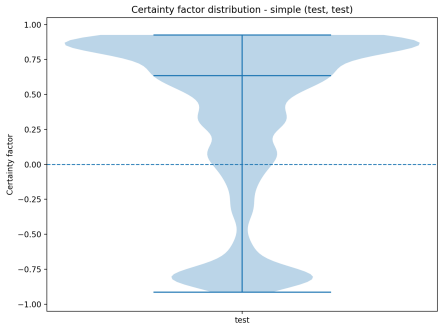

Figure 14: Violin plot of the certainty-factor distribution for the best Simple architecture (XXYY, 6 layers) in the noiseless test setting.

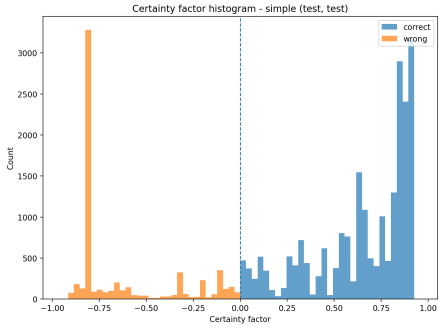

Figure 15: Histogram of the certainty-factor distribution for the best Simple architecture (XXYY, 6 layers) in the noiseless test setting.

The statistics computed separately on correctly and incorrectly classified samples provide additional insight into how sharp each architecture is. For QCNN, the mean signed output is 0.7534 on correct predictions and -0.8593 on incorrect ones. MERA and TTN show a very similar pattern, with mean values of 0.7161 and -0.8383 for MERA, and 0.7164 and -0.8156 for TTN. The large magnitudes observed for both groups indicate that these architectures tend to produce outputs far from zero not only when they are correct,

but also when they are wrong. This suggests that their predictions are often highly decisive, rather than merely accurate.

By contrast, the best Simple model exhibits smaller magnitudes on both subsets, with mean signed outputs of 0.6511 for correct samples and -0.6307 for incorrect ones. Its predictions are, on average, less extreme, even though the model achieves better overall classification performance. This comparison further supports the idea that certainty should be interpreted as a measure of decisiveness, not as a direct proxy for predictive quality.

3.4. Noisy Simulation and Certainty-Factor Degradation

After identifying the six-layer `XXYY` Simple architecture as the best noiseless configuration, the analysis was extended to noisy simulation to assess robustness under increasing noise. However, a full noisy evaluation of this model was computationally infeasible. For this reason, a lighter study was carried out within the same architectural family, using a two-layer `XXYY` Simple model. This choice made it possible to perform a complete sweep across multiple noise levels while still preserving the main structural characteristics of the selected architecture.

The noisy evaluation considers six regimes: *clean*, *very low*, *low*, *medium*, *high*, and *very high*. These correspond to **depolarizing-noise** probabilities of 0, 0.001, 0.005, 0.01, 0.05, and 0.1, respectively, applied to both single-qubit and two-qubit gates.

Table 8: Classification performance on the Simple architecture under increasing noise.

Noise level	Acc.	F1
Clean	0.7628	0.7748
Very low	0.7640	0.7790
Low	0.7680	0.7824
Medium	0.7560	0.7715
High	0.5600	0.6000
Very high	0.4930	0.5301

The results reveal a staged degradation pattern. In the clean, **very-low**, **low**, and **medium** noise regimes, the performance remains broadly stable: the F1-score ranges from 0.7715 to 0.7824, while accuracy varies between 0.7560 and 0.7680. Recall also remains above 0.80 throughout this region, and precision changes only moderately. These small variations should not be interpreted as a genuine benefit of noise. At low noise levels, the perturbation is generally too weak to affect the most confident predictions, as a result, only a limited subset of borderline cases changes label, which leads to slight metric fluctuations while the overall classification behavior remains essentially unchanged.

Table 9: Classification performance of the noisy study on the Simple architecture under increasing noise.

Noise level	Precision	Recall	ROC-AUC
Clean	0.7391	0.8079	0.7968
Very low	0.7389	0.8238	0.7935
Low	0.7433	0.8257	0.7892
Medium	0.7318	0.8158	0.7833
High	0.5546	0.6535	0.5780
Very high	0.4983	0.5663	0.4936

A qualitatively different regime emerges at **high** and **very-high** noise. At high noise, the F1-score drops to 0.6000, the accuracy to 0.5600, and the ROC-AUC to 0.5780. Under very high noise, performance approaches **near-random behavior**, with F1-score 0.5301, accuracy 0.4930, and ROC-AUC 0.4936. Therefore, weak and moderate perturbations do not immediately destroy thresholded classification, but sufficiently strong noise clearly undermines the reliability of the model.

Table 10: Certainty-factor statistics under increasing noise: average certainty magnitude and fraction of low-certainty samples.

Noise	Mean $\lvert C(x)\rvert$	Frac. $\lvert C\rvert < 0.2$	Frac. $\lvert C\rvert < 0.5$
Clean	0.6807	0.065	0.234
Very low	0.6425	0.066	0.250
Low	0.5074	0.093	0.429
Medium	0.3826	0.159	0.731
High	0.1156	0.788	1.000
Very high	0.1127	0.801	0.998

The **certainty-factor statistics**10 provide a clearer view of the degradation process. As noise increases, the average absolute certainty decreases steadily, from 0.6807 in the clean setting to 0.6425 under very low noise, 0.5074 under low noise, and 0.3826 under medium noise. This indicates that, even when hard-label metrics remain relatively stable, the model outputs are already moving closer to zero and therefore becoming less decisive.

This trend is also reflected in the **fraction of low-certainty samples**. The proportion of test examples with $|C(x)| < 0.2$ grows from 0.065 in the clean case to 0.066, 0.093, and 0.159 as noise increases from very low to medium. At the same time, the fraction with $|C(x)| < 0.5$ rises more sharply, from 0.234 in the clean regime to 0.250, 0.429, and 0.731. This shows that noise progressively pushes a larger portion of predictions toward the borderline region around the decision threshold, even before a strong drop in classification performance becomes visible.

At high and very high noise, this effect becomes extreme. The mean absolute certainty falls to about 0.11, while nearly all samples satisfy $|C(x)| < 0.5$ and about 80% fall below $|C(x)| < 0.2$. In this regime, the score distribution has largely collapsed toward zero, meaning that the classifier has lost most of its decisiveness, and this is reflected in the sharp degradation of predictive performance.

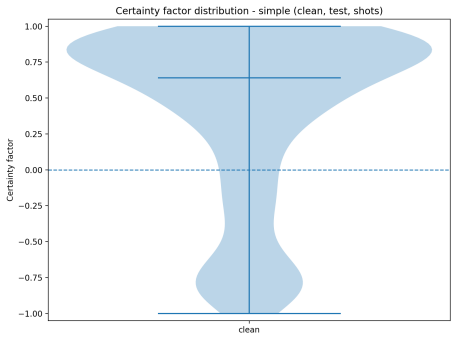

Figure 16: Violin plot of the certainty-factor clean setting. The distribution is broad and clearly separated from zero, indicating mostly confident predictions and a relatively small fraction of borderline samples.

Figures 16–22 provide a visual summary of this phenomenon through violin plots of the certainty-factor distribution. In the clean and very-low regimes, the distributions remain broad and clearly displaced away from zero. This indicates that most predictions still retain a substantial margin from the decision boundary.

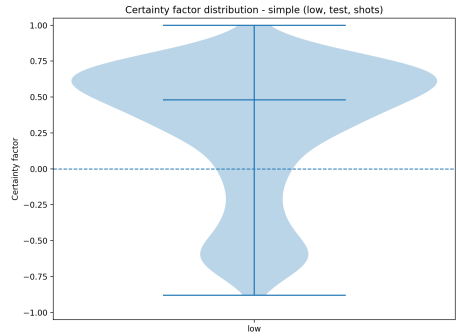

Figure 17: Violin plot of the certainty-factor under low noise. The distribution is still clearly displaced from zero, but already appears narrower than in the clean and very-low regimes, indicating reduced confidence margins.

In the **low-noise** regime**??**, the violin plot becomes visibly narrower, indicating that the certainty values are less extreme than in the clean case. However, the distribution remains clearly separated from zero, meaning that most predictions still retain the same sign and therefore the hard-label metrics change only marginally. This suggests that noise first reduces the decisiveness of the model before causing a substantial drop in classification performance.

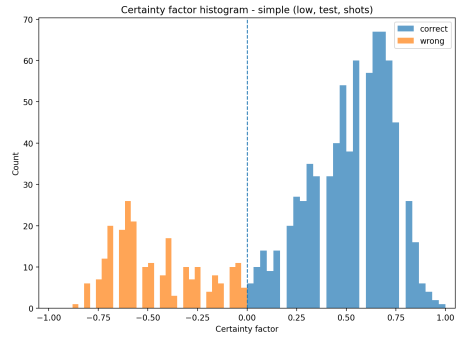

Figure 18: Histogram of the certainty-factor under low noise.

By the **medium** regime19, the contraction becomes much more evident. The violin plot is noticeably more concentrated toward the center, reflecting the strong increase in the number of intermediate-confidence predictions. This is fully consistent with the numerical result that 73.1% of the samples already satisfy $|C(x)| < 0.5$ at medium noise. Thus, even though F1-score and accuracy still remain reasonably strong, the certainty distribution reveals that the model is already losing decision sharpness.

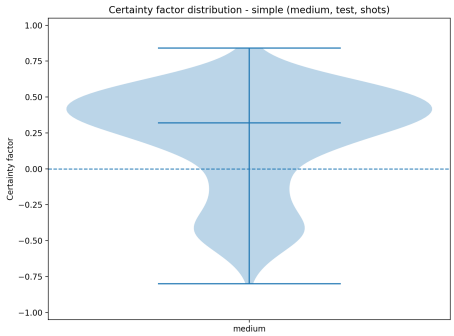

Figure 19: Violin plot of the certainty-factor under medium noise. The distribution becomes visibly compressed toward zero, showing that noise has already reduced decision sharpness even though the classification metrics remain relatively stable.

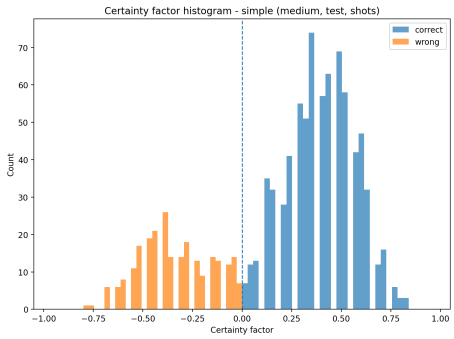

Figure 20: Histogram of the certainty-factor under medium noise.

The **high**- and **very-high-noise** violin plots show the final stage of degradation. In both cases, the distribution collapses into a narrow spindle around zero, indicating that the classifier is no longer producing sharply separated outputs. The very-high plot is especially revealing, as almost the entire mass is concentrated close to the decision boundary. This behavior is precisely the one expected from the reference paper: increasing noise squeezes the certainty-factor distribution toward the middle and gradually destroys the model's discriminative ability.

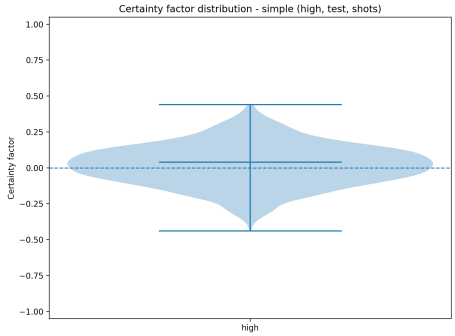

Figure 21: Violin plot of the certainty-factor under high noise. The distribution collapses toward the decision boundary, reflecting the strong loss of confidence and the clear degradation of classification performance.

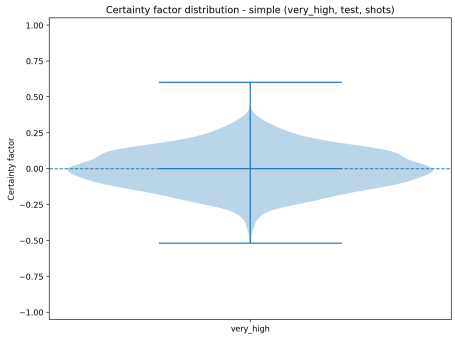

Figure 22: Violin plot of the certainty-factor under very high noise. Almost the entire distribution is concentrated near zero, indicating near-random behavior and a severe collapse of decision sharpness.

Overall, these experiments show that the impact of noise is progressive rather than abrupt. At weak and moderate noise, the classifier can still preserve a reasonable F1-score, but its certainty distribution is already being compressed toward zero, meaning that confidence and class separability are degrading before the final labels fail. At strong noise levels, this loss of margin becomes severe enough to produce a sharp decline in ROC-AUC, accuracy, and F1-score, eventually

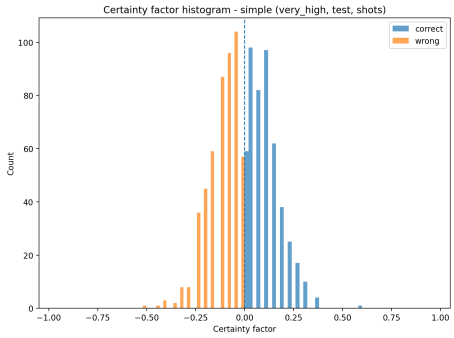

Figure 23: Histogram of the certainty-factor under very high noise.

leading to near-random behavior. Although this study uses a computationally cheaper two-layer XXYY proxy rather than the final six-layer best model, it already provides strong evidence for the main qualitative conclusion of the noisy analysis: increasing quantum noise progressively destroys decision sharpness, increases the number of borderline predictions, and ultimately undermines anomaly-detection performance.

4. Conclusion

This work investigated the use of quantum neural networks for network anomaly detection by comparing several architectures in both noiseless and noisy simulation settings. The analysis considered not only standard classification metrics, but also the certainty factor, in order to better characterize the confidence and separability of the model outputs.

The results suggest that quantum neural networks can provide meaningful performance for network anomaly detection, at least in the ideal simulation setting, with the best Simple configuration achieving the strongest overall classification performance among the tested models. At the same time, the comparison across architectures shows that sharper or more polarized outputs do not necessarily correspond to better anomaly-detection performance. In this respect, the certainty-factor analysis provided a useful complementary view of model behavior beyond standard accuracy-based metrics.

The noisy experiments also make clear how far current practice still is from robust real-world deployment. The noisy analysis shows that performance degradation is not only a matter of final accuracy, but also of a progressive loss of confidence and separation in the model outputs. In this sense, the certainty factor proved to be a useful tool: it offered a more nuanced view of model behavior and made the effect of noise visible even before the classification metrics collapsed.

Overall, the study shows that quantum models can be effective for this task in ideal simulation, while also making clear that noise has a substantial impact on their reliability. The obtained results therefore provide both a comparative assessment of the tested architectures and a more interpretable view of how their predictions degrade under realistic non ideal conditions.

References

[1] A. Kukliansky, M. Orescanin, C. Bollmann, and T. Huffmire. Network anomaly detection using quantum neural networks on noisy quantum computers. *IEEE Transactions on Quantum Engineering*, 5:1–11, 2024.

[2] M. Sarhan, S. Layeghy, and M. Portmann. Nf-unsw-nb15-v2, 2023. Dataset.